

Willkommen zur Dokumentation des Texture-Atlas-Projekts von Niklas Herrmann Kay Thomas, das im Rahmen des Computergrafik-Kurses an der Humboldt-Universität entstand. Wir erläutern die grundlegende Struktur des Projekts sowie dessen Inhalte, Assets, Bibliotheken und weitere Ressourcen; dabei werden zahlreiche Bilder veranschaulichend eingesetzt.

Ziele: Ursprünglich bestand die Idee darin, einen relativ einfachen Textur-Atlas zu erstellen, der sowohl Bodenbeläge (grassforesttile.png und jungleforesttile.png) als auch ein einfaches Farnmodell (fern.png) umfasst. Dabei kamen CMake, C++ und OpenGL zum Einsatz, wobei gleichzeitig Erfahrungen im Umgang mit bisher unbekanntem Code gesammelt wurden.

Dieses Projekt demonstriert das Sprite-Rendering in OpenGL unter Verwendung eines Textur-Atlas. Die Quellbilder werden in einer JSON-Datei beschrieben, in einer einzigen Atlas-Textur zusammengefasst und anschließend als texturierte Rechtecke (Quads) gezeichnet. Jedes Rechteck nutzt UV-Koordinaten, um den korrekten Teilbereich aus dem Atlas abzurufen.

Die Architektur ist bewusst einfach gehalten:

main.cpp initialisiert GLFW/OpenGL, lädt den Atlas und zeichnet die Sprites.

Shader.{h,cpp} kapselt das Laden und Kompilieren der Shader sowie die Aktualisierung der Uniform-Variablen.

shader.vert.txt und shader.frag.txt definieren das GLSL-Programm.

TextureAtlas.{h,cpp} erstellt die Atlas-Textur aus den in der JSON-Datei beschriebenen Bereichen.

Der Rendering-Ablauf gestaltet sich wie folgt:

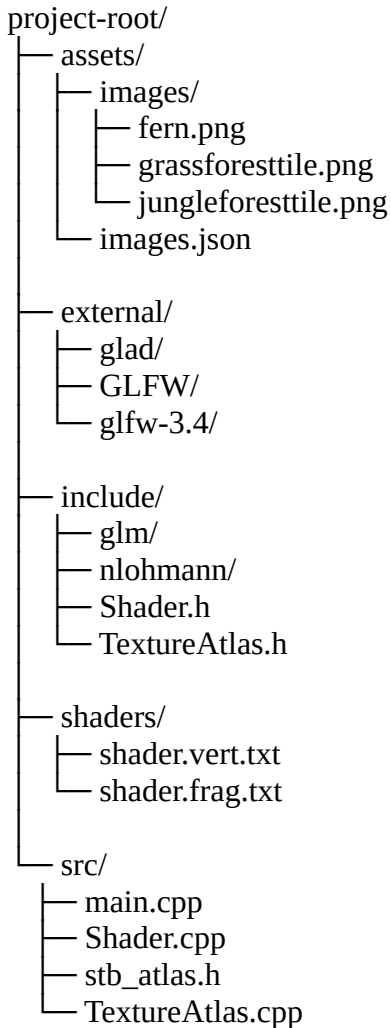
Laden der JSON-Datei mit Bildpfaden und Ausschnitts-Rechtecken.

Zusammenfassen dieser Rechtecke zu einer großen Textur (dem Atlas).

Auswählen eines benannten Atlas-Bereichs für jedes Sprite und Zeichnen eines Rechtecks unter Verwendung der entsprechenden UV-Koordinaten.

Der Fragment-Shader greift auf den Atlas zu und verwirft nahezu transparente Pixel, um saubere Ausschnitte zu erzielen.

Struktur des Projektverzeichnisses



Explanation of Codes and their functions:

main.cpp

Die Datei main.cpp bildet den Einstiegspunkt der Anwendung. Sie erstellt das Fenster sowie den OpenGL-Kontext, initialisiert GLAD, aktiviert das Blending, erzeugt das Shader-Programm, erstellt ein wiederverwendbares Quad, lädt und generiert den Textur-Atlas und führt schließlich die Render-Schleife aus, die in jedem Frame alle Sprites zeichnet.

Das Programm ist als minimales, aber vollständiges Beispiel für das OpenGL 3.3 Core-Profil aufgebaut. Der Großteil der Logik ist in kleine Hilfsfunktionen ausgelagert, wodurch die Hauptschleife übersichtlich und leicht erweiterbar bleibt.

Die main.cpp initialisiert die Anwendung, lädt den Textur-Atlas aus einer JSON-Datei und rendert eine Szene aus texturierten Quads. Sie richtet GLFW und OpenGL ein, kompiliert das Shader-Programm, erstellt ein einzelnes Quad, das für alle Sprites verwendet wird, und zeichnet anschließend jedes Sprite, indem sie die Position und die UV-Koordinaten des Quads an den jeweils gewählten Bereich des Atlas anpasst.

images.json

TextureAtlas.cpp(und .h)

TextureAtlas nimmt eine Liste von Quellbildern sowie Zuschneide-Rechtecke aus einer JSON-Datei entgegen, packt diese Rechtecke in eine einzige große Textur (den Atlas), lädt sie auf die GPU hoch und speichert für jeden Bereich normalisierte UV-Koordinaten. Der Renderer verwendet anschließend diese UV-Koordinaten, um jedes Sprite aus dem entsprechenden Teil des Atlas zu zeichnen.

TextureAtlas erstellt aus mehreren Quellbildern eine einzige OpenGL-Textur. Dabei werden die Quellbilder geladen, die angegebenen Bereiche zugeschnitten, diese in einem Atlas zusammengefasst und für jeden Bereich normalisierte UV-Koordinaten berechnet. Der Renderer nutzt diese UV-Koordinaten, um beim Zeichnen der Sprites den korrekten Teilbereich aus dem Atlas abzurufen.

TextureAtlas.cpp implementiert die Erstellung des Atlas. Dabei wird jedes Quellbild mit stb_image geladen, überprüft, ob die Zuschneide-Rechtecke innerhalb der Bildgrenzen liegen, und die Bereiche werden zeilenweise angeordnet; anschließend werden die zugeschnittenen Pixel in einen einzigen Puffer kopiert und das Ergebnis in eine OpenGL-Textur hochgeladen. Schließlich werden für jeden Bereich normalisierte UV-Koordinaten berechnet, damit der Shader das korrekte Rechteck innerhalb des Atlas abtasten kann.

Shader.cpp

Shader.cpp implementiert die in Shader.h deklarierte Shader-Klasse. Sie liest Shader-Quelldateien ein, kompiliert die Vertex- und Fragment-Shader, verknüpft diese zu einem Programm und stellt Methoden zur Verwendung des Programms sowie zur Aktualisierung von Uniforms bereit. Bei Fehlern während der Kompilierung oder Verknüpfung werden Fehlermeldungen auf der Konsole ausgegeben.

Shader.cpp implementiert die Shader-Klasse. Sie lädt GLSL-Quellcode von der Festplatte, kompiliert beide Shader-Stufen, verknüpft ein Programm und stellt Hilfsfunktionen zur Aktivierung des Programms sowie zur Aktualisierung von Uniform-Werten zur Verfügung. Dadurch wird die Shader-Einrichtung zentralisiert und die Fehlersuche erleichtert.

Shader.h

Shader.h deklariert eine kleine C++-Klasse, die ein OpenGL-Shader-Programm kapselt. Sie bietet:

Konstruktion anhand von Dateipfaden für Vertex- und Fragment-Shader.

Einen Destruktor, der das Programm löscht.

Methoden zur Aktivierung des Programms sowie zum Setzen von Integer- und Matrix-Uniforms.

Dies sorgt dafür, dass die Shader-Verwaltung gekapselt und wiederverwendbar bleibt.

Shader.h deklariert einen schlanken Wrapper für ein OpenGL-Shader-Programm. Die Klasse Shader lädt GLSL-Quellcode aus Dateien, kompiliert Shader, linkt ein Programm und stellt Hilfsmethoden zur Verwendung des Programms sowie zum Setzen von Uniforms bereit.

Shader.vert.txt

Dies ist der Vertex-Shader in GLSL. Er empfängt für jeden Vertex eine 2D-Position sowie 2D-UV-Koordinaten, leitet die UV-Koordinaten an den Fragment-Shader weiter und schreibt die Position direkt in den Clip-Space. Da diese Version weder eine Kamera noch eine Projektionsmatrix verwendet, liegen alle Positionen im normalisierten Bildschirmraum vor.

Der Vertex-Shader empfängt pro Vertex eine 2D-Position und 2D-UV-Koordinaten. Er gibt die UV-Koordinaten an den Fragment-Shader weiter und schreibt die Position direkt in ``gl_Position``, wobei von Koordinaten im normalisierten Bildschirmraum ausgegangen wird.

Shader.frag.txt

Dies ist der Fragment-Shader. Er tastet die Atlas-Textur mithilfe der interpolierten UV-Koordinaten ab, verwirft nahezu transparente Pixel für saubere Ausschnitte und schreibt die abgetastete Farbe in den Framebuffer. Genau dies verleiht Ihren Farnen und Gräsern ihr transparentes, nicht rechteckiges Erscheinungsbild.

Der Fragment-Shader tastet die Atlas-Textur unter Verwendung der interpolierten UV-Koordinaten ab. Er verwirft Fragmente mit sehr niedrigem Alpha-Wert, um transparente Cutout-Sprites zu ermöglichen, und gibt anschließend die abgetastete Farbe aus.

Bibliotheken Verwendet:

Dieses Projekt basiert auf den folgenden externen Bibliotheken:

GLFW

Plattformübergreifende Verwaltung von Fenstern und OpenGL-Kontexten. Übernimmt die Fenstererstellung sowie die Verarbeitung von Eingaben und Ereignissen.

GLAD

OpenGL-Funktions-Loader für modernes OpenGL (3.3 Core Profile). Stellt zur Laufzeit Funktionszeiger für alle verwendeten OpenGL-Aufrufe bereit.

GLM

OpenGL-Mathematikbibliothek. Bietet Vektor- und Matrixtypen sowie mathematische Hilfsfunktionen für C++, die an GLSL angelehnt sind.

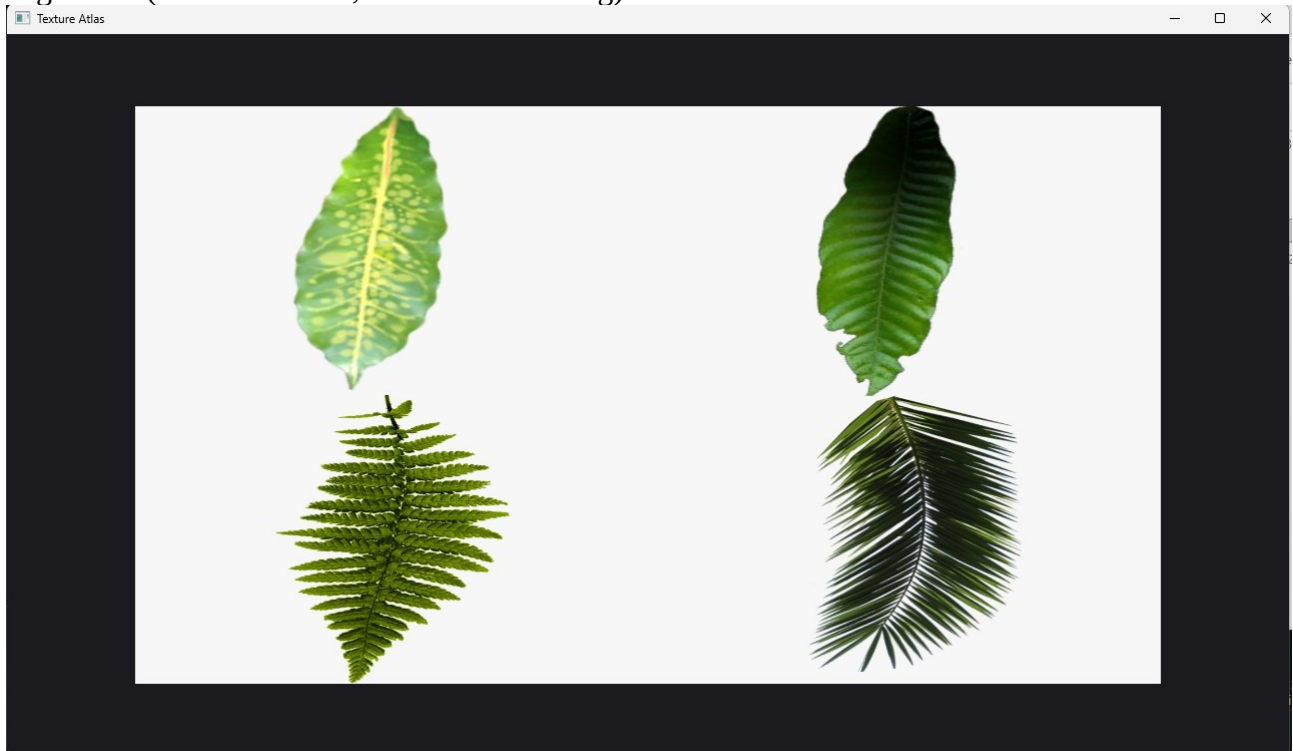
nlohmann/json

Moderne „Header-only“-JSON-Bibliothek für C++. Wird verwendet, um die Datei ``assets/images.json`` zu parsen und die Atlas-Konfiguration zu laden.

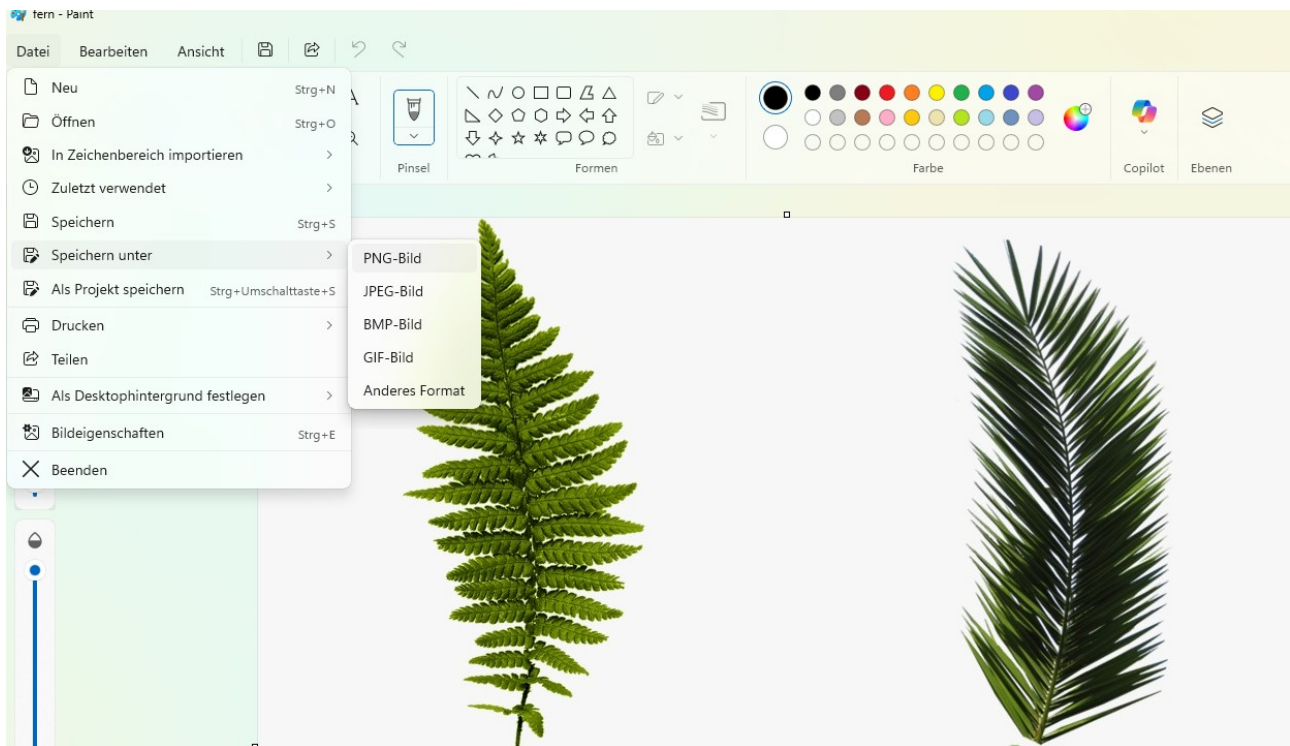
stb_image

Bild-Loader, der aus einer einzigen Header-Datei besteht (eingebunden über ``src/stb_atlas.h``). Lädt PNG-, JPEG- und andere gängige Bildformate und wandelt sie für das Atlas-Packing in Roh-Pixeln um.

Ergebnisse(Zuerst das Bild, dann die Erklärung):



Das erste funktionale Fenster, das 2D-Bilder aus einer „fern.png“-Datei verwendete – noch bevor ich mich mit Transparenz und deren Anwendung auf spätere Bilder befasste. Es wirkt etwas grob, und die Bilder waren riesig (512x512 Pixel in einem 2048x2048-Fenster, wodurch im Grunde kein Platz mehr für die Darstellung anderer Elemente blieb).



Ich habe etwas über Transparenz gelernt und festgestellt, dass PNG-Dateien in dem Projekt, an dem ich arbeitete, besser bzw. nahtloser geladen werden.

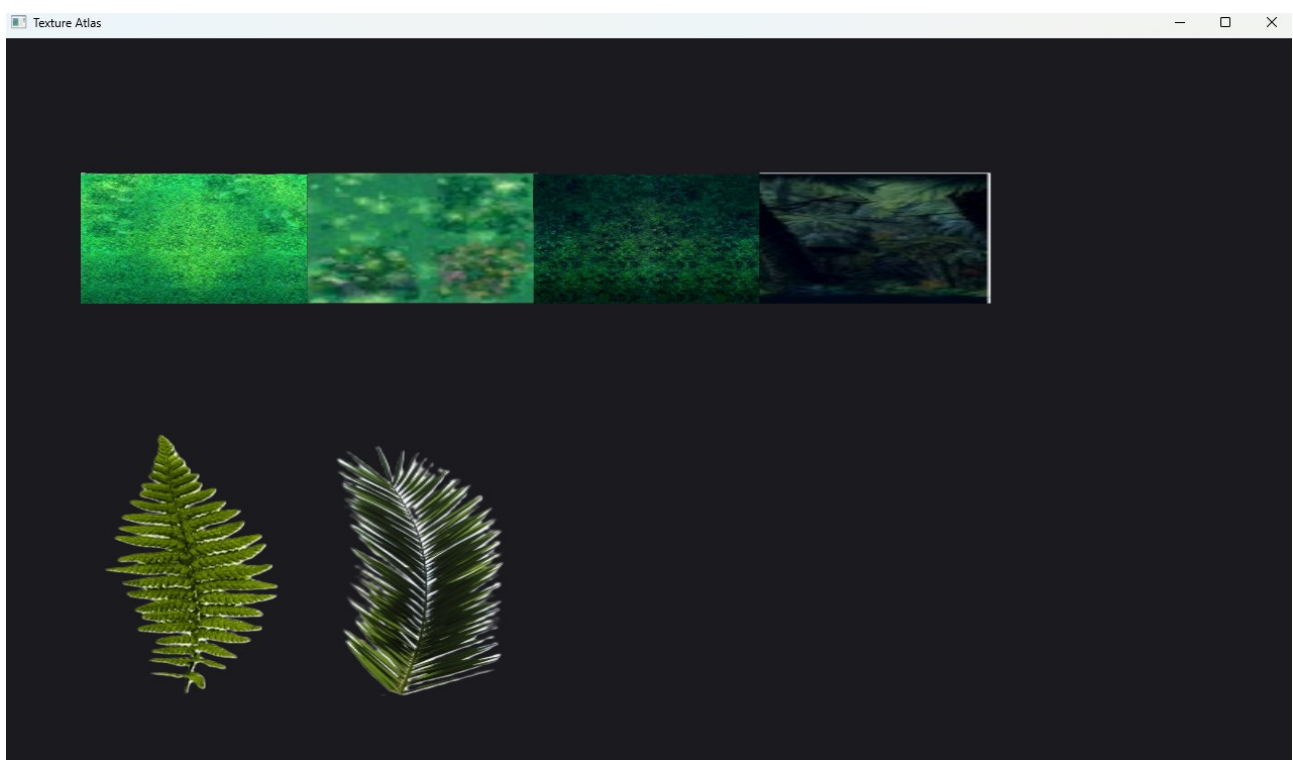


Probleme mit gespiegelten Bildern wurden durch Hinzufügen eines "Spiegelcodes" behoben, der jedoch veraltet ist, nachdem ich einige Probleme mit meinem shader.vert behoben hatte (der die Bilder aus einem mir unerklärlichen Grund spiegelte).



```
main.cpp | images.json | TextureAtlas.h | TextureAtlas.cpp | Shader.h | Shader.cpp | CMakeLists.txt | shade
assets > {} images.json > {} regions > {} 5
1 {
2   "images": [
3     "assets/images/fern.png",
4     "assets/images/grassforesttile.png",
5     "assets/images/jungleforesttile.png"
6   ],
7   "regions": [
8     { "source": "assets/images/grassforesttile.png", "x": 68, "y": 0, "w": 256, "h": 257, "name": "grass_large" },
9     { "source": "assets/images/grassforesttile.png", "x": 2, "y": 132, "w": 64, "h": 64, "name": "grass_small" },
10    { "source": "assets/images/jungleforesttile.png", "x": 68, "y": 0, "w": 256, "h": 257, "name": "jungle_large" },
11    { "source": "assets/images/jungleforesttile.png", "x": 3, "y": 67, "w": 64, "h": 127, "name": "jungle_small" },
12    { "source": "assets/images/fern.png", "x": 55, "y": 0, "w": 156, "h": 259, "name": "multileaf_bright" },
13    { "source": "assets/images/fern.png", "x": 319, "y": 0, "w": 145, "h": 256, "name": "multileaf_dark" }
14  ]
15 }
```

Dies ist die JSON-Datei, die die Verknüpfungen zu den Speicherorten der Bilder herstellt, damit der Code sie laden kann.



Verschiedene Bilder aus unterschiedlichen Dateien werden auf einem einzigen Textur-Atlas angezeigt.

Ein großes 10x10-Raster, das eine Schleife der zuvor gezeigten Bilder anzeigt; der Code lädt es relativ schnell und ist einsatzbereit.

```

static void buildGridSprites(std::vector<Sprite>& sprites) {
    sprites.clear();

    const std::vector<std::string> names = {
        "grass_large",
        "grass_small",
        "jungle_large",
        "jungle_small",
        "multileaf_bright",
        "multileaf_dark"
    };

    const int cols = 10;
    const int rows = 10;

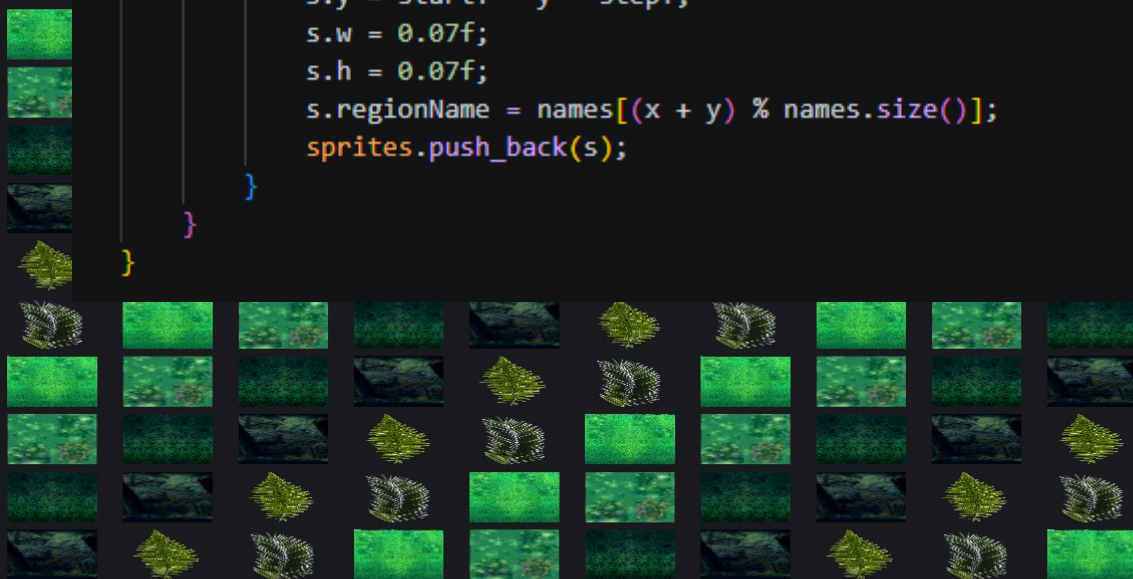
    const float startX = -0.90f;
    const float startY = 0.80f;
    const float stepX = 0.18f;
    const float stepY = 0.16f;

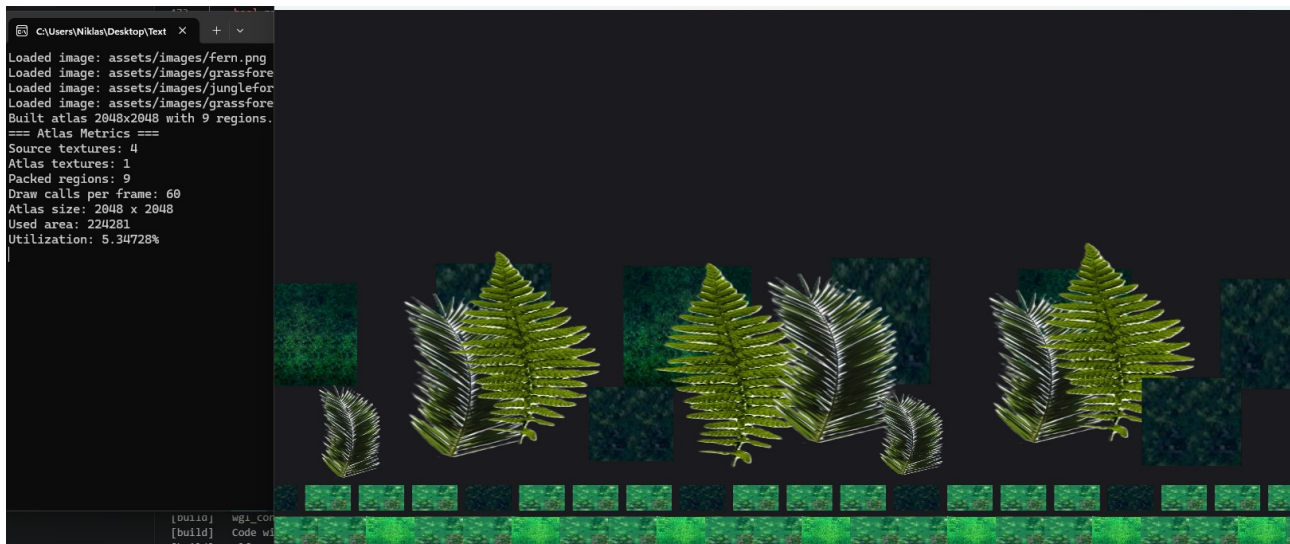
    for (int y = 0; y < rows; ++y) {
        for (int x = 0; x < cols; ++x) {
            Sprite s;
            s.x = startX + x * stepX;
            s.y = startY - y * stepY;
            s.w = 0.07f;
            s.h = 0.07f;
            s.regionName = names[(x + y) % names.size()];
            sprites.push_back(s);
        }
    }
}

```

Texture At

×





Das Endergebnis des Codes: Einige Bilder werden im Vordergrund und andere im Hintergrund angezeigt, dazu ein großer Bodenbereich mit mehreren Sprites:

Quelltexturen

Anzahl der geladenen Original-Bilddateien (z. B. fern.png, grassforestile.png, jungleforestile.png). Dies zeigt an, wie viele Dateien zu einem einzigen Atlas zusammengefasst wurden.

Atlas-Texturen

Hier fest auf 1 gesetzt, da Ihr Code eine kombinierte GL_TEXTURE_2D-Textur erstellt. Es geht darum zu bestätigen, dass Sie viele Quellbilder auf eine einzige GPU-Textur reduziert haben.

Gepackte Bereiche (Packed regions)

Anzahl der AtlasRegion-Einträge im Atlas (d. h. wie viele einzelne Sprites/UV-Rechtecke Sie namentlich ansprechen können).

In Ihrer JSON-Datei entspricht dies der Anzahl der Einträge unter „regions“.

Draw-Calls pro Frame

Anzahl der glDrawArrays- oder glDrawElements-Aufrufe, die Sie pro Frame für Sprites tätigen. In Ihrer Schleife verursacht jedes Sprite einen Draw-Call; der Wert entspricht also sprites.size(). Dies ist ein wichtiger Leistungsindikator: Weniger Draw-Calls bedeuten normalerweise eine bessere GPU-Auslastung.

Atlas-Größe

Breite und Höhe (in Pixeln) der kombinierten Textur, z. B. 2048 x 2048.

Größere Atlanten können mehr Inhalt aufnehmen, verschwenden aber Speicherplatz, wenn sie nicht voll ausgenutzt werden.

Genutzte Fläche

Summe aus srcW * srcH für alle gepackten Bereiche:

$\text{size_t usedArea} = \sum (\text{region.srcW} * \text{region.srcH})$

Dies ist die gesamte „Inhaltsfläche“, die für Sie relevant ist – ohne Berücksichtigung von Padding (Abständen) oder Leerraum.

Auslastung/Utilization (%)

Verhältnis der genutzten Fläche zur gesamten Atlas-Fläche:

$$\text{utilization} = (\text{usedArea}/(\text{atlasW} \times \text{atlasH})) \times 100\%$$

Dies gibt Aufschluss darüber, wie effizient Sie den zugewiesenen Texturspeicher nutzen.

Eine niedrige Auslastung (z. B. <30 %) deutet darauf hin, dass der Atlas zu groß ist oder viel ungenutzter Raum vorhanden ist; eine sehr hohe Auslastung (>85–90 %) kann auf eine sehr dichte Packung mit wenig Spielraum für Erweiterungen hinweisen.

Git Link: <https://github.com/Herrmnik/Niklas-Texture-Atlas>